

— 실습 구성

01 Step 0: 환경 진단

02 Step 1: 핵심 시나리오 + E2E 테스트 작성

03 Step 2: 자율 디버깅 루프

04 Step 3: scenario-verifier + 검증 루프

05 Step N: 정리와 비용 비교

06 과제 + Tips

STEP 0

환경 진단

자료 6 산출물 점검과 Playwright MCP 연결 확인

- 01 자료 6 산출물 5개 점검 (화면 2개 + sub-agent 2개 + hook 2개)
- 02 Playwright MCP 연결 확인
- 03 /cost 토큰 베이스라인 메모
- 04 다음 Step 진입 준비 완료

0-1. 환경 확인 명령

운영체제별 도구 버전 + planning_project 진입

MACOS / LINUX

```
node --version
claude --version
npx playwright --version
cd planning_project
```

WINDOWS (POWERSHELL)

```
node --version
claude --version
npx playwright --version
Set-Location planning_project
```

planning_project 폴더에서 작업해야 자료 6 산출물이 그대로 보입니다.

0-2. 자료 6 산출물 점검

비어 있는 항목 있으면 자료 6 해당 Step으로 돌아가 보강

#	산출물	점검
1	화면 2개 동작	카피 입력 + 카피 결과
2	<code>.claude/agents/prd-reviewer.md</code>	있는가
3	<code>.claude/agents/git-committer.md</code>	있는가
4	<code>settings.json</code> 의 <code>hooks</code> (<code>PostToolUse</code> + <code>Stop</code>)	등록됐는가
5	Issue #2-#10 PR 병합	됐는가

비어 있는 항목 있으면 자료 6의 해당 Step으로 돌아가 보강 후 본 단계 진입.

0-3. 토큰 베이스라인 + 0-4. Playwright MCP 연결

0-3. 토큰 베이스라인

/cost

현재 토큰 사용량 메모. 본 sprint 종료 시 비교 대상.

0-4. PLAYWRIGHT MCP 연결 확인

Playwright MCP가 연결되어 있는지 확인해줘.

연결 안 되어 있으면 MCP 설정 추가:

claude mcp add playwright

Step 0 통과 기준

5개 항목 모두 통과 시 Step 1 (E2E 테스트 작성)로 진입.

- ☐ `planning_project` 폴더에서 작업 중인가
- ☐ 자료 6 산출물 5개가 모두 있는가
- ☐ Playwright가 설치되어 있는가
- ☐ Playwright MCP가 연결되어 있는가
- ☐ /cost 베이스라인을 메모했는가

— STEP 1

핵심 시나리오 + E2E 테스트 작성

PRD 핵심 시나리오 1개를 자연어로 E2E 테스트로 변환

- 01 docs/prd.md에서 핵심 시나리오 1개 선택
- 02 자연어 명령으로 E2E 테스트 작성 위임
- 03 Playwright MCP로 테스트 코드 작성 + 실행
- 04 통과 → Step 3, 실패 → Step 2 분기 진입

1-1. PRD 핵심 시나리오 선택

사용자가 처음부터 끝까지 거치는 흐름 1개

정의

[docs/prd.md](#) 에서 사용자가 처음부터 끝까지 거치는 흐름 1개.

시드 6번 사례

셀러가 카피 입력 화면에 상품 사진과
키워드 5개를 입력하고 '카피 생성' 버튼을 클릭하면
결과 화면에 차별화 카피 3개가 표시되고,
복사 버튼을 클릭하면 카피 1개가 클립보드에 복사된다

사용자가 PRD 가설을 검증하는 핵심 흐름이 시나리오의 첫 후보.

1-2. 자연어 E2E 테스트 명령

Claude Code 세션에 시나리오 + **Playwright MCP** 위임

docs/prd.md의 핵심 시나리오를 E2E 테스트로 만들어줘:

"셀러가 카피 입력 화면에 상품 사진과 키워드 5개를 입력하고
'카피 생성' 버튼을 클릭하면 결과 화면에 차별화 카피 3개가 표시된다"

Playwright MCP로 테스트 코드 작성과 실행까지 해줘.
테스트 파일은 e2e/ 폴더에 뒀어.

Playwright 는 E2E 자동화 표준 도구, **MCP** 는 Claude가 직접 실행하는 채널.

테스트 코드를 직접 작성 안 함. 시나리오만 자연어로. 위임 3원칙이 테스트에도 적용.

— 활동 1-3 · 결과 분기

1-3. 테스트 실행 결과 확인

통과

→ Step 3 (시나리오 검증)으로

실패

→ Step 2 (자율 디버깅)으로

분기 결과에 따라 다음 Step 자동 선택.

활동 1-검증 · STEP 1 통과 기준

Step 1 검증 기준

3개 항목 모두 통과 시 Step 2 또는 Step 3 진입

- ☐ E2E 테스트 코드가 `e2e/` 폴더에 작성됐는가
- ☐ 테스트 실행이 시작됐는가 (통과 또는 실패 무관)
- ☐ 시나리오가 PRD 핵심 가설을 검증하는가

— STEP 2

자율 디버깅 루프

테스트 실패 시 자율 디버깅 3원칙 적용해 통과까지 도달

- 01 자율 디버깅 명령 (3원칙 적용)
- 02 명령 점검: 3원칙 (에러 그대로 / 분석 먼저 / 결과 검증)
- 03 에이전트 디버깅 흐름 5단계 확인
- 04 무한 루프 방지: 정리 명령 + 시드6 사례

2-1. 자울 디버깅 명령 (3원칙 적용)

테스트 실패 시 Claude Code 세션 명령

다음 E2E 테스트 실패 메시지를 분석해주세요:

[에러 전체 메시지를 그대로 붙여넣기]

실패 원인을 코드의 어느 부분에서 찾을 수 있는지
분석한 뒤, 수정 방안을 제시하고 적용해주세요.
수정 후 테스트를 재실행해 통과 여부를 확인해주세요.

3원칙: 1) 에러 그대로 / 2) 분석 먼저 / 3) 결과 검증

명령 점검: 3원칙

원칙	적용 위치
1. 에러 그대로	"전체 메시지를 그대로 붙여넣기"
2. 분석 먼저	"분석한 뒤, 수정 방안 제시하고 적용"
3. 결과 검증	"재실행해 통과 여부 확인"

에이전트 디버깅 흐름 확인

- 1 에러 분석 결과 출력
- 2 수정 방안 제시
- 3 코드 수정 적용
- 4 테스트 재실행
- 5 통과 또는 추가 수정 필요

활동 2-3 · 무한 루프 방지

무한 루프 방지: 정리 명령

같은 자리 3회 이상 맴돌면 정리 명령

이 디버깅의 어디서 막혔는지 정리해줘.
지금까지 시도한 수정과 각 수정 후의 에러를
표로 보여줘.

정리된 표 보고 작업자가 직접 판단. 에이전트가 못 푸는 문제는 작업자가 직접 보기.

시드 6번 흔한 디버깅 사례 + Step 2 검증

시드 6번 흔한 실패 3건

- [실패 1] 결과 화면에 카피 0개 표시**

원인: OpenAI API 호출 실패 또는 응답 파싱 오류

수정: API 키 환경변수 확인 + 응답 구조 점검
- [실패 2] 라우팅 후 빈 화면**

원인: 입력값이 query 파라미터로 안 넘어감

수정: router.push에 query 추가
- [실패 3] 키워드 5개 검증 통과 안 됨**

원인: 빈 문자열도 5개로 카운트

수정: filter로 빈 문자열 제외 후 length 체크

Step 2 검증 기준 (4항목 모두 통과 시 Step 3 진입)

- ☐ 자율 디버깅 3원칙 명령을 사용했는가
- ☐ 에이전트가 분석·수정·재실행 흐름을 따랐는가
- ☐ 테스트가 통과했는가 (또는 막힘 메모가 있는가)
- ☐ 무한 루프에 빠진 경우 정리 명령을 사용했는가

— STEP 3

scenario-verifier 작성과 시나리오 검증 루프

새 sub-agent 작성과 docs/prd.md 기반 시나리오 검증 루프 진행

- 01 scenario-verifier.md 작성 (6원칙 적용)
- 02 첫 호출 + 보고서 3단 구조 확인
- 03 검증 루프 진행 (검증 → 수정 → 재검증)
- 04 통과 조건 + 작업자 본인 검증

3-1. scenario-verifier.md 작성

.claude/agents/scenario-verifier.md 파일 작성

```
---
name: scenario-verifier
description: 빌드 완료 후 호출.
  docs/prd.md의 핵심 시나리오 1개와 전체 코드를 비교해
  시나리오가 처음부터 끝까지 통과 가능한지 검증합니다.
tools: Read, Grep, Glob
model: sonnet
---

시나리오 단위 PRD 검증 전문가의 역할.

**입력**: 검증할 시나리오 설명 1개

**작업 절차**:
1. docs/prd.md를 Read로 읽고 시나리오 흐름 확인
2. Grep/Glob으로 시나리오 관련 파일 식별
3. 각 단계가 코드로 구현됐는지 추적
4. 화면 간 라우팅과 데이터 전달 점검
5. 막힘 지점이 있으면 구체적 위치 보고

**출력 형식** (3단 구조):
- 시나리오 통과 여부: 통과 / 부분 통과 / 미통과
- 막힘 지점: 어느 단계가 어디서 막히는지 (파일 줄)
- 수정 제안: 무엇을 추가/수정해야 하는지

**주의**:
- 수정 권한 없음 (tools에 Write/Edit 없음)
- 추측 금지. docs/prd.md와 실제 코드만 보고 판단
- 보고서는 한국어
```

Agent 설계 6원칙 모두 적용. 단일 책임(시나리오 검증), 최소 권한(Read/Grep/Glob), 결정적 출력(3단), 책임 분리(수정 권한 없음), 호출 시점(빌드 완료 후), 입출력 계약(본문에 명시).

3-2. scenario-verifier 첫 호출

@scenario-verifier 명시 호출 + 시나리오 + docs/prd.md 점검 위임

@scenario-verifier 다음 시나리오를 검증해줘:

"셀러가 카피 입력 화면에 상품 사진과 키워드 5개를 입력하고
'카피 생성' 버튼을 클릭하면 결과 화면에 차별화 카피 3개가
표시되고, 복사 버튼을 클릭하면 카피 1개가 클립보드에 복사된다"

docs/prd.md를 읽고 현재 코드가 이 시나리오를
처음부터 끝까지 동작시킬 수 있는지 점검해줘.

명시 호출(@scenario-verifier) — 자동이 아닌 작업자 호출 시점.

시드 6번 scenario-verifier 첫 보고서

3단 구조: 통과 여부 / 막힘 지점 / 수정 제안

[시나리오 통과 여부] 부분 통과	[막힘 지점] 단계 1 (입력): <code>src/components/InputForm.tsx</code> 정상 동작 (통과) 단계 2 (생성): <code>src/app/api/generate/route.ts</code> 에서 OpenAI 응답을 <code>JSON.parse</code> 실패 시 에러 처리 누락 단계 3 (복사): <code>src/components/CopyButton.tsx</code> 에서 <code>navigator.clipboard</code> 호출만 있고 실패 시 fallback 없음
[수정 제안] 1. route.ts에 try-catch와 에러 응답 추가 2. CopyButton에서 clipboard 실패 시 textarea fallback	

3-2. 검증 루프 진행

검증 → 작업자 확인 → 수정 → 재검증 → 통과

- 1 @scenario-verifier 첫 호출 → 보고서
- 2 작업자가 막힘 지점 1-2개 직접 확인 — 보고서가 가리킨 파일·줄 열어보기 / 실제로 문제가 있는지 검증
- 3 수정 코드 작성 (위임 3원칙 + 자료 6의 자동 hook 작동)
- 4 다시 @scenario-verifier 호출
- 5 통과까지 반복

루프의 본질: scenario-verifier는 검증만, 수정은 메인 에이전트. 책임 분리.

3-3. 검증 우선주의 적용

sub-agent 보고도 AI 산출물 — 작업자 본인 검증 필수

점검 항목	방법
막힘 지점 진위	보고서가 가리킨 파일·줄 직접 확인
수정 제안 적절성	docs/prd.md 와 본인이 대조
자동 신뢰 금지	sub-agent 보고도 AI 산출물

3-4. 통과 조건 (AND 조건 3)

세 조건 모두 충족 시 Step 3 완료

조건 1

보고서 시나리오 통과 여부: "통과"

AND

조건 2

작업자가 막힘 지점 0건 직접 확인

AND

조건 3

E2E 테스트도 통과 (Step 1-2 결과와 일치)

활동 3-검증 · STEP 3 통과 기준

Step 3 검증 기준

7개 항목 모두 통과 시 Step N 진입

- ☐ `.claude/agents/scenario-verifier.md` 파일이 작성됐는가
- ☐ 검증 루프를 1회 이상 돌았는가 (검증 → 수정 → 재검증)
- ☐ frontmatter가 6원칙을 따랐는가
- ☐ 최종 보고서가 "통과"인가
- ☐ 첫 호출 보고서가 3단 구조로 출력됐는가
- ☐ 작업자의 본인 검증으로 통과 확정인가
- ☐ 막힘 지점을 작업자가 직접 확인했는가

— 활동 N-1 · 산출물 정리

Step N: 정리와 비용 비교

본 단계 목표는 산출물 확인과 토큰 비교, 회고

N-1. 산출물 정리

#	산출물	다음 단계 활용
1	통과되는 E2E 테스트 1개	회귀 테스트
2	scenario-verifier sub-agent	시나리오 변경 시 재검증
3	자율 디버깅 경험 1회	다음 에러에 적용
4	시나리오 검증 보고서 누적	통과 증거
5	검증된 MVP 상태	배포 진입

— 활동 N-2 · 비용 비교

N-2. /cost 비교

본 sprint 종료 시 토큰 사용량 비교

/cost

sub-agent 컨텍스트 격리 덕분에 토큰 효율. scenario-verifier 호출 1회당 보통 2,000-5,000 토큰.

N-3. 회고: 좋은 답 vs 나쁜 답

구체적·근거 있는 답이 좋은 답

질문	좋은 답	나쁜 답
자율 디버깅 3원칙 중 가장 효과적	"분석 먼저. 추측 수정 차단됨"	"다 좋았다"
E2E 통과 시점 안도감	"라우팅 문제로 3번 막혔다가 통과 시 시나리오가 한 흐름으로 작동"	"기뻐다"
배포 진입 자신감	"scenario-verifier가 통과 보고 + E2E 통과 → 두 증거로 진입"	"준비됐다"

— 활동 N-검증 · STEP N 통과 기준

Step N 검증 기준

4개 항목 모두 통과 시 본 sprint 완료

- ☐ 산출물 5개가 모두 손에 있는가
- ☐ `/cost` 비교를 메모했는가
- ☐ 회고 3개 질문에 모두 답했는가
- ☐ 다음 단계(배포) 진입 준비가 됐는가

활동 · 검증된 MVP 배포 진입

과제

본 자료의 과제는 `assignment.md` 파일에 정리되어 있다.

항목	내용
목표	검증된 MVP 상태로 배포 단계 진입
산출물	통과되는 E2E 테스트, <code>scenario-verifier</code> , 자율 디버깅 경험, 시나리오 통과 보고서
마감	다음 학습 단계 시작 전

E2E 통과 + `scenario-verifier` 통과 두 증거가 손에 있어야 배포 단계 진입 가능.

— TIP 1

테스트 단계 팁

동작의 진짜 정의와 두 증거

테스트 단계 팁

Tip 제목

시나리오 통과가 동작의 진짜 정의

화면 뺐다고 통과 아님. 사용자가 처음부터 끝까지 흐름 완수해야 통과.

Tip 제목

E2E 통과 + scenario-verifier 통과 두 증거

E2E는 실행 증거, **scenario-verifier** 는 PRD 일치 증거. 두 증거가 모두 있어야 다음 단계.

— TIP 2

Playwright MCP 팁

E2E 자연어 위임과 사전 확인

Playwright MCP 팁

Tip 제목

E2E 테스트 코드도 직접 안 씀

시나리오만 자연어로 말함. 위임 3원칙이 테스트 작성에도 적용.

Tip 제목

Playwright MCP 연결 사전 확인

Step 0에서 미리 확인. 연결 안 되어 있으면 `claude mcp add playwright` .

— TIP 3

자율 디버깅 3원칙 팁

에러 그대로 / 분석 먼저 / 결과 검증의 보편화

자율 디버깅 3원칙 팁

Tip 제목

모든 에러에 적용

한 번 익히면 본 학습 후에도 평생 사용. AI 협업의 보편 패턴.

Tip 제목

에러 메시지는 정보의 보고

길고 무서워 보여도 그대로 복사. 요약 금지. 스택 트레이스가 디버깅의 핵심.

Tip 제목

무한 루프 차단

같은 자리 3회 이상 맴돌면 "어디서 막혔는지 정리해줘"부터.

— TIP 4

scenario-verifier 팁

책임 분리와 검증 루프

scenario-verifier 팁

Tip 제목

prd-reviewer와 책임 분리

prd-reviewer는 코드 단위(자동), **scenario-verifier** 는 시나리오 단위(명시 호출). Agent 설계 단일 책임 원칙.

Tip 제목

Manyfast MCP 사용 안 함

docs/prd.md 를 Read로 직접 읽음. 외부 MCP 의존 없이 작동. 자료 4-6과 일관.

Tip 제목

검증은 단발이 아닌 루프

검증 → 수정 → 재검증을 통과까지 반복. 한 번 호출하고 결과 그대로 수용 금지.

— TIP 5

검증 우선주의 팁

sub-agent 보고도 AI 산출물

— 활동 TIP 5 · 검증 우선주의

검증 우선주의 팁

sub-agent 보고도 AI 산출물

"통과"라고 적힌 보고서도 본인이 검증. 막힘 지점 1-2개를 직접 파일 열어 확인.